Exp No: 7	**Deep Recurrent Neural Network for Image Caption Generation Combining a CNN with an RNN**

AIM:

To build a deep recurrent neural network that generates image captions by combining a CNN for image feature extraction with an RNN for sequence generation.

ALGORITHM / PROCEDURE:

1. Create a captioned image dataset in code: 12x12 RGB images, each with one coloured shape (4 colours, 3 shapes) in one of 4 positions, with random jitter and noise. The caption is `a <colour> <shape> at the <position>`, using a vocabulary of 16 words. 7 of the 48 colour-shape-position combinations are kept out of training and used only for testing.
2. **CNN encoder:** two blocks of 3x3 convolution + ReLU + 2x2 max pooling (8 and 16 filters), then a dense layer that gives a 64-value image feature vector.
3. **Deep RNN decoder:** the image feature sets the start state of two stacked tanh RNN layers (64 units each).
4. At every step the previous word is converted to a 16-value embedding and passed through RNN layer 1 and RNN layer 2, and a softmax layer predicts the next word. The input is `<start>` followed by the caption words, and the target is the caption words followed by `<end>` (teacher forcing).
5. Train the whole model end to end with cross-entropy loss, using backpropagation through time through both RNN layers and into the CNN (Adam, lr = 0.003, batch size 32, 40 epochs, gradient clipping at 5). All gradients were verified against numerical gradients.
6. **Caption generation:** encode the image, start from `<start>` and repeatedly pick the most likely word, feeding it back as the next input.
7. Measure the exact-caption accuracy and the colour, shape and position accuracy on test images with combinations seen and not seen in training.

Fig 1: CNN + deep RNN captioning model

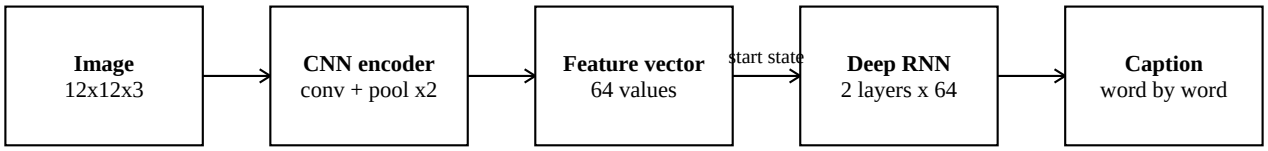
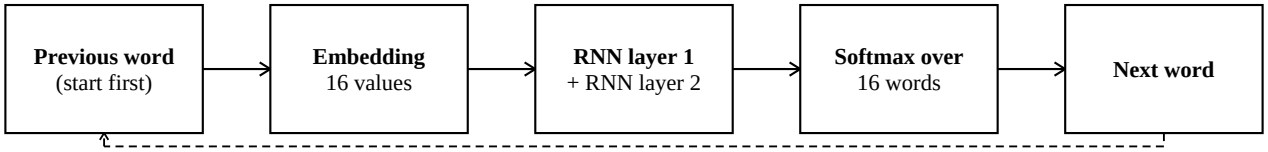


Fig 2: One step of caption generation



Commands / Settings used:

Component	Details
Dataset	Generated in code: 12x12 RGB images, 16-word vocabulary
Images	1312 train, 328 test (seen combinations), 140 test (unseen combinations)
Encoder	Conv 3x3 (8) - Conv 3x3 (16), each with ReLU + max pool, dense 64
Decoder	Embedding 16, two stacked tanh RNN layers of 64 units, softmax
Training	Cross-entropy, Adam lr 0.003, batch 32, 40 epochs

PROGRAM:

```
import textwrap
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from numpy.lib.stride_tricks import sliding_window_view as swv

rng = np.random.default_rng(0)

# ---------- Dataset: images of coloured shapes with captions (generated in code) ----------
COLORS = {"red": (1, 0, 0), "green": (0, 1, 0), "blue": (0, 0, 1), "yellow": (1, 1, 0)}
SHAPES = ["square", "circle", "triangle"]
POS = {"top left": (3, 3), "top right": (3, 9), "bottom left": (9, 3), "bottom right": (9, 9)}
VOCAB = (["<start>", "<end>", "a", "at", "the"] + list(COLORS) + SHAPES
         + ["top", "bottom", "left", "right"])
W2I = {w: i for i, w in enumerate(VOCAB)}
V, T = len(VOCAB), 8                          # vocabulary size, caption length (steps)
yy, xx = np.mgrid[0:12, 0:12]


def draw(color, shape, pos):
    img = rng.uniform(0, 0.15, (12, 12, 3))                      # noisy background
    cy, cx = np.array(POS[pos]) + rng.integers(-1, 2, 2)         # random jitter
    if shape == "square":
        mask = (yy >= cy - 2) & (yy <= cy + 1) & (xx >= cx - 2) & (xx <= cx + 1)
    elif shape == "circle":
        mask = (yy - cy) ** 2 + (xx - cx) ** 2 <= 4.5
    else:
        r = yy - (cy - 2)
        mask = (r >= 0) & (r <= 3) & (np.abs(xx - cx) <= (r + 1) // 2)
    img[mask] = np.array(COLORS[color]) * rng.uniform(0.7, 1.0)
    return img


def caption(color, shape, pos):
    return ["a", color, shape, "at", "the"] + pos.split()


combos = [(c, s, p) for c in COLORS for s in SHAPES for p in POS]
def code(cb):
    return list(COLORS).index(cb[0]) + SHAPES.index(cb[1]) + list(POS).index(cb[2])


unseen = [cb for cb in combos if code(cb) % 6 == 0]       # combinations kept out of training
seen = [cb for cb in combos if cb not in unseen]


def make(cbs, n):
    imgs, caps = [], []
    for cb in cbs:
        for _ in range(n):
            imgs.append(draw(*cb))
            caps.append([W2I[w] for w in caption(*cb)])
    return np.array(imgs), np.array(caps)
Xs, Cs = make(seen, 40)
```

```
perm = rng.permutation(len(Xs))
ntr = int(0.8 * len(Xs))
Xtr, Ctr = Xs[perm[:ntr]], Cs[perm[:ntr]]                # training images
Xts, Cts = Xs[perm[ntr:]], Cs[perm[ntr:]]                # test: seen combinations
Xtu, Ctu = make(unseen, 20)                              # test: NEW combinations
print(f"Vocabulary: {V} words | combinations: {len(seen)} seen, {len(unseen)} unseen")
print(f"Images: {len(Xtr)} train, {len(Xts)} test (seen), {len(Xtu)} test (unseen combos)")


# ---------- CNN encoder (image feature extraction) ----------
def im2col(x, k):
    p = k // 2
    return swv(np.pad(x, ((0, 0), (p, p), (p, p), (0, 0))), (k, k), axis=(1, 2))


class Conv:
    def __init__(self, cin, cout, k=3):
        self.k = k
        self.W = rng.standard_normal((cin * k * k, cout)) * np.sqrt(2 / (cin * k * k))
        self.b = np.zeros(cout)
        self.gW, self.gb = np.zeros_like(self.W), np.zeros_like(self.b)

    def forward(self, x):
        self.shape = x.shape
        N, H, W, C = x.shape
        self.cols = im2col(x, self.k).reshape(N * H * W, -1)
        return (self.cols @ self.W + self.b).reshape(N, H, W, -1)

    def backward(self, g):
        N, H, W, C = self.shape
        k, p = self.k, self.k // 2
        g2 = g.reshape(N * H * W, -1)
        self.gW, self.gb = self.cols.T @ g2, g2.sum(0)
        dc = (g2 @ self.W.T).reshape(N, H, W, C, k, k)
        dxp = np.zeros((N, H + 2 * p, W + 2 * p, C))
        for i in range(k):
            for j in range(k):
                dxp[:, i:i + H, j:j + W, :] += dc[:, :, :, :, i, j]
        return dxp[:, p:p + H, p:p + W, :]


class Dense:
    def __init__(self, nin, nout):
        self.W = rng.standard_normal((nin, nout)) * np.sqrt(2 / nin)
        self.b = np.zeros(nout)
        self.gW, self.gb = np.zeros_like(self.W), np.zeros_like(self.b)

    def forward(self, x):
        self.x = x
        return x @ self.W + self.b

    def backward(self, g):
        self.gW, self.gb = self.x.T @ g, g.sum(0)
        return g @ self.W.T


class ReLU:
    def forward(self, x):
        self.m = x > 0
        return x * self.m

    def backward(self, g):
        return g * self.m


class MaxPool2:
    def forward(self, x):
        N, H, W, C = x.shape
        self.xr = x.reshape(N, H // 2, 2, W // 2, 2, C)
        self.out = self.xr.max(axis=(2, 4))
        return self.out

    def backward(self, g):
        m = self.xr == self.out[:, :, None, :, None, :]
        m = m / m.sum(axis=(2, 4), keepdims=True)
```

```
        N, h, _, w, _, C = self.xr.shape
        return (m * g[:, :, None, :, None, :]).reshape(N, 2 * h, 2 * w, C)


class Flatten:
    def forward(self, x):
        self.shape = x.shape
        return x.reshape(len(x), -1)

    def backward(self, g):
        return g.reshape(self.shape)


FEAT, HID, EMB = 64, 64, 16
encoder = [Conv(3, 8), ReLU(), MaxPool2(), Conv(8, 16), ReLU(), MaxPool2(),
           Flatten(), Dense(3 * 3 * 16, FEAT), ReLU()]


def encode(x):
    for l in encoder:
        x = l.forward(x)
    return x


def encode_back(g):
    for l in reversed(encoder):
        g = l.backward(g)


def enc_pairs():
    return [(a, getattr(l, n)) for l in encoder if hasattr(l, "W")
            for a, n in ((l.W, "gW"), (l.b, "gb"))]


# ---------- Deep RNN decoder: 2 stacked tanh RNN layers (sequence generation) ----------
def init(nin, nout):
    return rng.standard_normal((nin, nout)) * np.sqrt(1 / nin)


R = {"E": rng.standard_normal((V, EMB)) * 0.3,
     # image feature -> start state of RNN layer 1 and layer 2
     "Wi1": init(FEAT, HID), "bi1": np.zeros(HID),
     "Wi2": init(FEAT, HID), "bi2": np.zeros(HID),
     "Wx1": init(EMB, HID), "Wh1": init(HID, HID), "b1": np.zeros(HID),
     "Wx2": init(HID, HID), "Wh2": init(HID, HID), "b2": np.zeros(HID),
     "Wo": init(HID, V), "bo": np.zeros(V)}


def start_states(f):
    return np.tanh(f @ R["Wi1"] + R["bi1"]), np.tanh(f @ R["Wi2"] + R["bi2"])


def rnn_step(tok, h1, h2):
    e = R["E"][tok]
    a1 = np.tanh(e @ R["Wx1"] + h1 @ R["Wh1"] + R["b1"])
    a2 = np.tanh(a1 @ R["Wx2"] + h2 @ R["Wh2"] + R["b2"])
    return e, a1, a2, a2 @ R["Wo"] + R["bo"]


def softmax(z):
    e = np.exp(z - z.max(-1, keepdims=True))
    return e / e.sum(-1, keepdims=True)


def loss_and_grads(imgs, caps):
    """Teacher forcing: input = <start> + words, target = words + <end>. Full BPTT."""
    N = len(imgs)
    f = encode(imgs)
    inp = np.concatenate([np.full((N, 1), W2I["<start>"]), caps], axis=1)
    tgt = np.concatenate([caps, np.full((N, 1), W2I["<end>"])], axis=1)
    h1s, h2s = [None], [None]
    h1s[0], h2s[0] = start_states(f)
    cache, probs = [], []
    for t in range(T):
        e, a1, a2, lg = rnn_step(inp[:, t], h1s[-1], h2s[-1])
```

```
        h1s.append(a1)
        h2s.append(a2)
        cache.append(e)
        probs.append(softmax(lg))
    loss = -np.mean([np.log(probs[t][np.arange(N), tgt[:, t]] + 1e-12) for t in range(T)])
    # ----- backward through time -----
    G = {k: np.zeros_like(v) for k, v in R.items()}
    dh1 = dh2 = np.zeros((N, HID))
    for t in range(T - 1, -1, -1):
        dl = probs[t].copy()
        dl[np.arange(N), tgt[:, t]] -= 1
        dl /= N * T
        a1, a2 = h1s[t + 1], h2s[t + 1]
        G["Wo"] += a2.T @ dl
        G["bo"] += dl.sum(0)
        dz2 = (dl @ R["Wo"].T + dh2) * (1 - a2**2)
        G["Wx2"] += a1.T @ dz2
        G["Wh2"] += h2s[t].T @ dz2
        G["b2"] += dz2.sum(0)
        dz1 = (dz2 @ R["Wx2"].T + dh1) * (1 - a1**2)
        G["Wx1"] += cache[t].T @ dz1
        G["Wh1"] += h1s[t].T @ dz1
        G["b1"] += dz1.sum(0)
        np.add.at(G["E"], inp[:, t], dz1 @ R["Wx1"].T)
        dh1, dh2 = dz1 @ R["Wh1"].T, dz2 @ R["Wh2"].T
    dz1 = dh1 * (1 - h1s[0] ** 2)
    dz2 = dh2 * (1 - h2s[0] ** 2)
    G["Wi1"], G["bi1"], G["Wi2"], G["bi2"] = f.T @ dz1, dz1.sum(0), f.T @ dz2, dz2.sum(0)
    encode_back(dz1 @ R["Wi1"].T + dz2 @ R["Wi2"].T)       # gradient flows into the CNN
    return loss, G


def generate(imgs):
    """Greedy decoding: feed each predicted word back in as the next input."""
    f = encode(imgs)
    h1, h2 = start_states(f)
    tok = np.full(len(imgs), W2I["<start>"])
    out = []
    for t in range(T):
        _, h1, h2, lg = rnn_step(tok, h1, h2)
        tok = lg.argmax(1)
        out.append(tok)
    return np.stack(out, axis=1)[:, :7]                    # the 7 caption words


def words(ids):
    return " ".join(VOCAB[i] for i in ids)


def evaluate(imgs, caps):
    g = generate(imgs)
    exact = np.mean((g == caps).all(1))
    colour = np.mean(g[:, 1] == caps[:, 1])
    shape = np.mean(g[:, 2] == caps[:, 2])
    pos = np.mean((g[:, 5:7] == caps[:, 5:7]).all(1))
    return [x * 100 for x in (exact, colour, shape, pos)], g


# ---------- Training (end to end, Adam) ----------
if __name__ == "__main__":
    params = [(a, None) for a in R.values()] + [(a, None) for a, _ in enc_pairs()]
    m = [np.zeros_like(a) for a, _ in params]
    v = [np.zeros_like(a) for a, _ in params]
    t, epochs, batch, lr = 0, 40, 32, 0.003
    print(f"\n{'Epoch':<8}{'Train loss':<13}{'Exact caption acc. (test, seen)'}")
    for ep in range(1, epochs + 1):
        idx = rng.permutation(len(Xtr))
        tot = 0.0
        for s in range(0, len(idx), batch):
            b = idx[s:s + batch]
            loss, G = loss_and_grads(Xtr[b], Ctr[b])
            tot += loss * len(b)
            grads = list(G.values()) + [g for _, g in enc_pairs()]
            norm = np.sqrt(sum((g**2).sum() for g in grads))
            sc = min(1.0, 5.0 / (norm + 1e-12))              # gradient clipping
```

```
        t += 1
        for i, ((a, _), g) in enumerate(zip(params, grads)):
            g = g * sc
            m[i] = 0.9 * m[i] + 0.1 * g
            v[i] = 0.999 * v[i] + 0.001 * g * g
            a -= lr * (m[i] / (1 - 0.9**t)) / (np.sqrt(v[i] / (1 - 0.999**t)) + 1e-8)
    if ep == 1 or ep % 5 == 0:
        acc = evaluate(Xts, Cts)[0][0]
        print(f"{ep:<8}{tot / len(Xtr):<13.4f}{acc:.2f}%")

# ---------- Evaluation ----------
(rs, gs), (ru, gu) = evaluate(Xts, Cts), evaluate(Xtu, Ctu)
print(f"\n{'Test set':<26}{'Images':<8}{'Exact':<9}{'Colour':<9}{'Shape':<9}{'Position'}")
print("-" * 68)
rows = (("Seen combinations", len(Xts), rs), ("Unseen combinations", len(Xtu), ru))
for name, n, r in rows:
    print(f"{name:<26}{n:<8}{r[0]:<9.2f}{r[1]:<9.2f}{r[2]:<9.2f}{r[3]:.2f}")

print("\nExample captions (generated | reference):")
pick = list(range(0, 6 * 20, 20))
for i in pick:
    print(f"  [unseen] {words(gu[i])[2:]:<36}| {words(Ctu[i])[2:]}")

# ---------- Figure: images with generated captions ----------
fig, ax = plt.subplots(2, 4, figsize=(8, 4.4))
for k, a in enumerate(ax.ravel()):
    src = (Xts, gs, Cts) if k < 4 else (Xtu, gu, Ctu)
    i = (k % 4) * 17 + 3
    a.imshow(np.clip(src[0][i], 0, 1), interpolation="nearest")
    ok = (src[1][i] == src[2][i]).all()
    cap = words(src[1][i])
    a.set_title(textwrap.fill(cap, 20), fontsize=7.5,
                color="green" if ok else "red")
    a.axis("off")
fig.text(0.01, 0.75, "Seen\ncombos", fontsize=8, va="center")
fig.text(0.01, 0.28, "Unseen\ncombos", fontsize=8, va="center")
plt.tight_layout(rect=(0.06, 0, 1, 1))
plt.savefig("img/exp7_captions.png", dpi=200)
```

OUTPUT:

Terminal - CNN + RNN image captioning

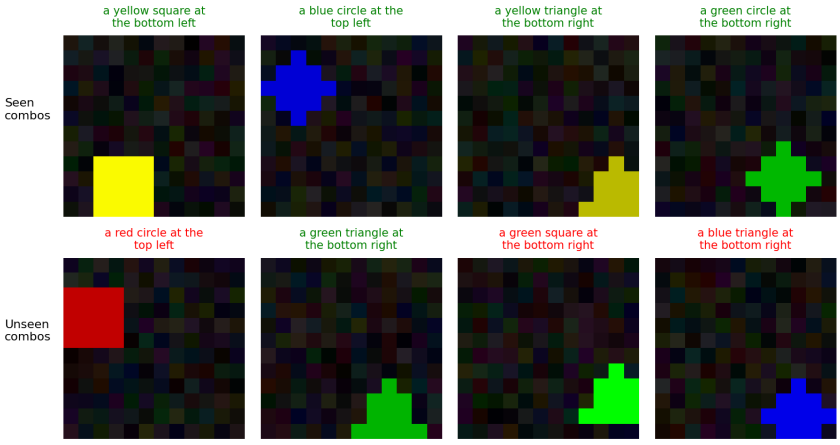
```
root@ip-10-10-5-10:~# python3 exp7.py
Vocabulary: 16 words | combinations: 41 seen, 7 unseen
Images: 1312 train, 328 test (seen), 140 test (unseen combos)

Epoch   Train loss   Exact caption acc. (test, seen)
1       0.7193       8.54%
5       0.1283       50.00%
10      0.0303       87.80%
15      0.0050       99.39%
20      0.0071       95.73%
25      0.0047       99.09%
30      0.0004       100.00%
35      0.0003       100.00%
40      0.0002       100.00%

Test set                 Images  Exact    Colour   Shape    Position
-----------------------------------------------------------------
Seen combinations        328     100.00   100.00   100.00   100.00
Unseen combinations      140     3.57     97.86    3.57     100.00

Example captions (generated | reference):
  [unseen] red circle at the top left          | red square at the top left
  [unseen] green triangle at the bottom right  | green triangle at the bottom right
  [unseen] blue triangle at the bottom right   | blue circle at the bottom right
  [unseen] blue circle at the bottom left      | blue triangle at the bottom left
  [unseen] yellow triangle at the bottom right | yellow square at the bottom right
  [unseen] yellow triangle at the bottom left  | yellow circle at the bottom left
```

Fig 3: Captions generated for test images (green = correct, red = wrong)



RESULT:

Thus an image captioning model with a CNN encoder and a two-layer RNN decoder was built from scratch and trained end to end. The training loss fell from 0.7193 to 0.0002 and all 328 test images with combinations seen in training received the exact correct caption (100%). On 140 images of 7 combinations never seen in training, the colour (97.86%) and position (100%) were right but the shape was right for only 3.57% of images, so exact captions were also 3.57%. The model therefore did not learn the shape independently of the colour and position combinations it saw in training. The dataset is a small generated one, not MS COCO.